

Backend Services

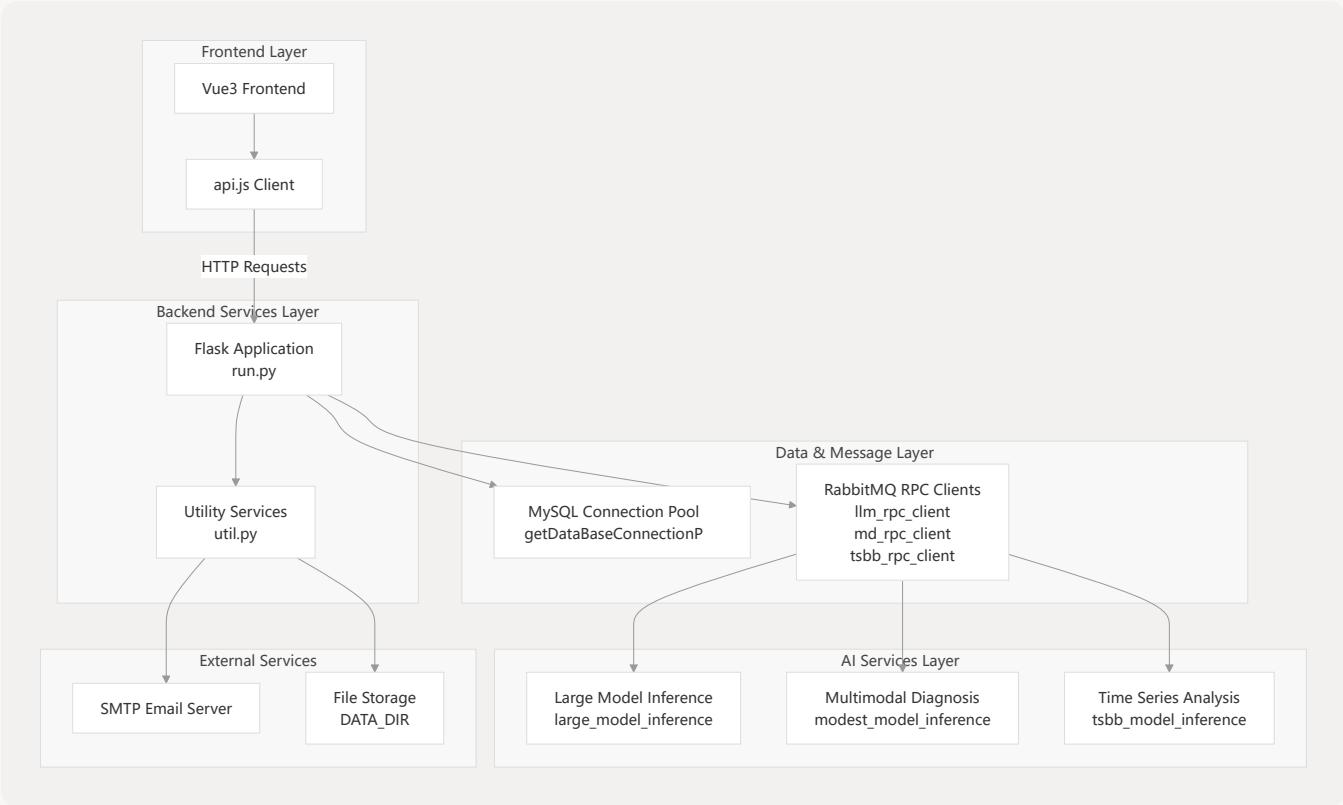
> Relevant source files

The backend services form the core API layer of the MKTY Smart Healthcare System, providing RESTful endpoints, data management, and integration with AI services. This document covers the overall backend architecture, Flask application structure, and infrastructure components. For specific API endpoint implementations, see [Core API Endpoints](#). For utility functions and infrastructure services, see [Utilities & Infrastructure](#).

Purpose and Architecture Overview

The backend services are implemented as a Flask-based REST API server that handles user authentication, medical data management, AI service orchestration, and system administration. The architecture follows a layered approach with clear separation between API endpoints, business logic, and infrastructure services.

System Architecture



Flask Application Core

The main Flask application serves as the central API server, handling HTTP requests, authentication, and business logic coordination.

Application Configuration

The Flask application is configured with essential middleware and global parameters:

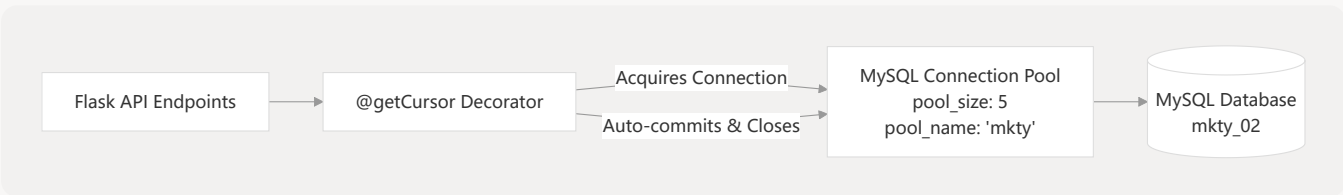
Configuration	Value	Purpose
MODE	'dev'	Runtime environment mode
PORT	5555	Backend service port
HOST	'0.0.0.0'	Service host address
JWT_SECRET_KEY	STRONG_PASSWORD	JWT token encryption
JWT_ACCESS_TOKEN_EXPIRES	False	Token expiration policy

The application initializes with CORS support for cross-origin requests and JWT manager for authentication.

Sources: backend/run.py 38-95

Database Integration

The backend uses MySQL with connection pooling for scalable database access:

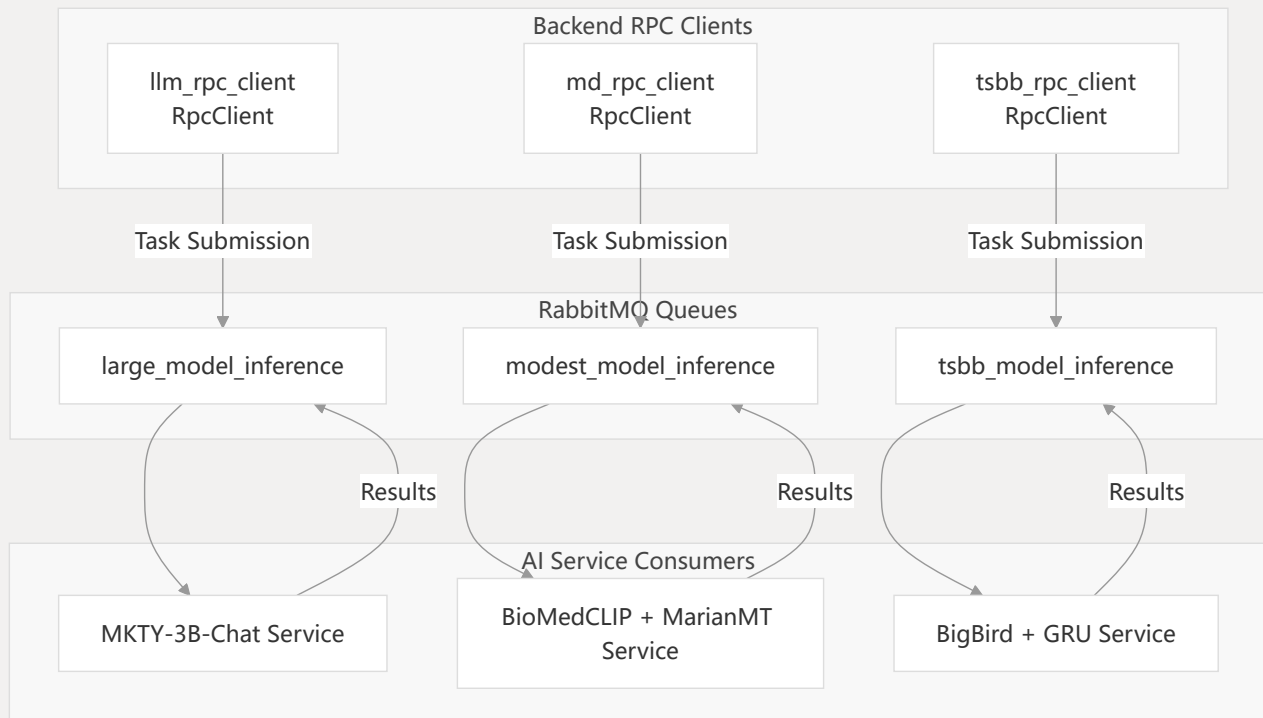


The `@getCursor` decorator automatically manages database connections, providing each endpoint with a configured cursor object and handling connection cleanup.

Sources: backend/run.py 68-83 backend/util.py 215-242

Message Queue Architecture

The backend communicates with AI services through RabbitMQ using custom RPC clients:



Each RPC client maintains its own connection parameters and callback queue for asynchronous task processing.

Sources: `backend/run.py` 50-88 `backend/util.py` 269-333

API Endpoint Categories

The backend provides comprehensive REST API endpoints organized by functional domain:

Authentication & User Management

- **Login/Registration:** `loginVerification`, `register`
- **Profile Management:** `getUserInfo`, `modifyUserInfo`, `modifyUserAvatar`, `modifyUserPassword`
- **Avatar Handling:** `getUserAvatar` with Base64 encoding

Medical & AI Services

- **LLM Inference:** `llmInferenceSubmitTask`, `llmInferenceGetStatus`
- **Multimodal Diagnosis:** `multimodalDiagnosisSubmitTask`, `multimodalDiagnosisGetStatus`
- **Time Series Analysis:** `tsbbModelSubmitTask`, `tsbbInferenceGetStatus`
- **Session Management:** `saveLlmSession`, `getLlmSession`, `deleteLlmSession`

Healthcare Data Management

- **Medical Records:** `getMedicalRecords`, `createMedicalRecord`, `updateMedicalRecord`
- **Important Tasks:** `addImportantItem`, `deleteImportantItem`, `finishImportantItem`
- **Knowledge Entities:** `createKnowledgeEntity`, `searchKnowledgeEntities`

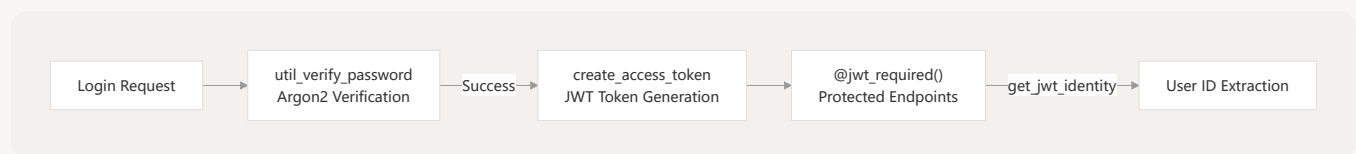
Communication & Community

- **Messaging:** `addMailItem`, `getMailList`, `deleteMailItem`
- **Forums:** `addForum`, `getForumList`, `deleteForum`, `sendPost`
- **Email Services:** `sendEmail`, `exportChatToPDF`

Sources: `backend/run.py` 103-1470 `frontend/src/api/api.js` 80-541

Security and Authentication

The backend implements JWT-based authentication with Argon2 password hashing:



Password encryption uses the Argon2 algorithm with default parameters, providing strong protection against rainbow table and brute force attacks.

Sources: `backend/run.py` 118-157 `backend/util.py` 244-267

File and Data Processing

The backend handles various file operations and data transformations:

Base64 File Handling

- **Encoding:** `util_file2base64` with MIME type detection
- **Decoding:** `util_base642file` with header removal
- **Image Processing:** `save_base64_image` with format conversion

Document Processing

- **Text Extraction:** Support for `.txt`, `.docx`, `.pptx`, `.pdf` files
- **Text Segmentation:** `split_text_into_pieces` for knowledge management
- **Search Features:** BM25-based Chinese text search with jieba tokenization

PDF Generation

The system can export chat conversations to formatted PDF documents using WeasyPrint with custom CSS styling.

Sources: `backend/util.py` 136-719

Infrastructure Services

Email Service

The backend provides SMTP email functionality with HTML content support, attachment handling, and SSL/TLS encryption. Configuration includes sender authentication and server parameters.

Database Connection Management

The `@getCursor` decorator pattern ensures proper database connection lifecycle management:

- Automatic connection acquisition from pool
- Transaction auto-commit for data consistency
- Guaranteed connection cleanup and release

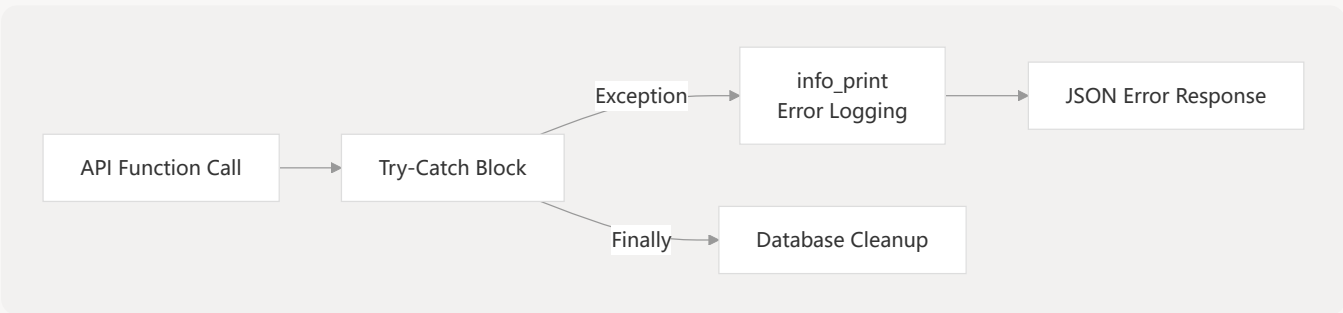
UUID and Timestamp Utilities

- **GUID Generation:** `util_uuid()` for unique identifiers
- **Time Management:** `util_current_time()` for Unix timestamp generation

Sources: `backend/util.py` 180-547

Error Handling and Logging

The backend implements comprehensive error handling with the `info_print` logging system:



Log levels include `info`, `warning`, `error`, and `initial` with color-coded console output using the Rich library.

Sources: backend/util.py 90-135